



🧠 Objectif du module

Il est d'usage de commencer par les variables. Toutefois pour une raison pratique je vais commencer par la fonction `printf()`. En effet, je voudrais que l'on puisse directement avoir un feedback lors de manipulation de données et un print est ce qui nous convient le mieux.

Nous allons survoler le principe des prints dans la console. Cela nous donne l'option d'avoir des "traces" qui nous permettront de lire et d'avoir un feedback à un moment donné dans le code sur une variable donnée.

Le print a quelques limites et est parfois illisible ou inutilisable dans une boucle d'update par exemple.

Le debugger offre un certain nombre d'outils plus avancés pour mettre en pause et avoir le temps de lire et suivre ses données. Toutefois, sa mise en place est hors du scope de cette formation et sera vue lors d'une formation future de niveau intermédiaire.

Mais le print reste un outil formidable pour le prototypage rapide et avoir un retour d'information dans la majorité des cas, même pour développer un projet commercial.

- Ce que l'élève saura faire à la fin : Utiliser `printf` pour afficher des variables dans la console et `DrawText` avec `TextFormat` dans raylib.
- Notions techniques abordées : Inclusion de bibliothèques, formatage avec specifiers, et prints dans

📖 Théorie

La méthode printf :

Nous allons utiliser une fonction qui s'appelle `printf`. Pour ce faire, nous devons inclure la bibliothèque `<stdio.h>`.

```
#include <stdio>
```

Je vous conseille vivement de parcourir les fonctions ou outils que vous fournissent chaque librairie que vous installez. Je vous renvoie au glossaire à la fin de ce pdf pour le lien descriptif.

L'inclusion consiste à lier le code actuelle avec soit :

- un élément du projet " " (vue plus tard lors de la séparation des éléments)
- ou du dossier d'installation standard entre < > (les bibliothèques de base fournies avec C++)

Et si vous ne comprenez rien à la lecture de la lib demandez à chatgpt ou autre IA de vous fournir une description synthétique de la librairie et de ses principaux outils. (je vous avoue que ça va plus vite mais faire les 2 en parallèle paye plus à long terme).

Nous verrons pas à pas comment l'utiliser dans le screen suivant.

The screenshot shows a C++ IDE with a file named `main.cpp`. The code is as follows:

```
1 #include "raylib.h"
2 #include <stdio>
3
4 int main(void)
5 {
6     // create a windows for the app with size and title as param
7     int testPrint = 10;
8     InitWindow(800, 450, "raylib [core] example - basic window");
9     while (!WindowShouldClose())
10     {
11         BeginDrawing();
12         printf("x = %d\n", testPrint);
13         // need to clear the background every frame see the tutorial on the hello world section
14         ClearBackground(RAYWHITE);
15         DrawText("Congrats! You created your first window!", 190, 200, 20, LIGHTGRAY);
16         EndDrawing();
17     }
18     CloseWindow();
19     return 0;
20 }
```

Annotations in the image:

- 1: Points to `#include <stdio>`.
- 2: Points to `int testPrint = 10;`.
- 3: Points to `printf("x = %d\n", testPrint);`.
- 4: Points to the `TERMINAL` tab in the IDE.
- 5: Points to the output in the terminal.

The terminal output shows:

```
x = 10
x = 10
x = 10
x = 10
x = 10
x = 10
```

Le formatage :

1. Inclusion de la bibliothèque nécessaire
2. Instantiation d'une variable entière pour le test
3. Print de la variable (pas d'inquiétude, on va voir le langage R2D2 ensemble)
4. Le terminal (je l'appelle console perso) qui nous donne les infos
5. Le print de la variable dans la console

Le **printf** en lui-même est relativement simple d'emploi, mais la syntaxe nécessaire est un peu rude. C'est pourquoi j'ai décidé d'en faire une carte afin que cela devienne rapidement une habitude. Nous allons nous concentrer sur les types de base. Voyons pour commencer sa structuration.

`%[flags][width][.precision][length]specifier`

- **flags** = options d'affichage (alignement, signe, remplissage...)
- **width** = largeur minimale
- **.precision** = précision (nb de décimales ou longueur max)
- **length** = modificateur de taille (l, ll, h, etc.)
- **specifier** = lettre qui indique le type

En fait, on ne se servira que du spécifier (le type), les autres paramètres ne sont pas nécessaires pour l'instant. Les types de base sont les suivants :

- %d ou %i → entier signé (int)
- %u → entier non signé (unsigned int)
- %c → caractère (char)
- %s → chaîne de caractères (char* terminé par \0)
- %f → nombre flottant décimal (ex: 3.141593)
- %p → adresse mémoire

Il faut encore penser à ajouter le \n qui renvoie à la ligne (mais c'est situationnel, cela dépend de l'affichage que vous voulez obtenir).

Exemple détaillé :

Reprenons notre exemple. Appel de la fonction **printf** et passage des paramètres suivants entre parenthèses :

```
printf("x = %d\n",testPrint);
```

- "x = " : mettre entre " " signifie une écriture de texte, on le voit bien dans la [console](#) (cf image précédente)
- %d : correspond donc à une variable de type entier signé (borne négative et positive +/-)
- \n : est ce que l'on appelle un retour chariot (sûrement d'une époque lointaine où les dinosaures faisaient de la dactylographie sur machine à écrire)
- Après la virgule se trouve la variable que l'on veut afficher, qui du coup doit être un int (je ne vais plus les appeler par le nom complet)
- Comme le tout est entre " ", le texte et la valeur seront transformés pour être affichés dans la console

Printf Personnalisé

La [console](#) peut parfois être surchargée et il est alors plus difficile de trouver les données. C'est pourquoi j'ai fourni quelques screens pour accrocher l'œil rapidement. Le plus simple étant de mettre une ligne vide \n après le print (cf image ci-dessous) ou avant et après, c'est encore plus visible. Ou personnalisez vos prints (peut-être même dans le futur créer une fonction personnalisée qui appelle le **printf** de la manière qui vous convient le mieux).

Printf avec un double saut de ligne

Double saut dans la console

```
printf("x = %d\n\n",testPrint);  
printf("x = %d\n\n",testPrint);  
printf("x = %d\n\n",testPrint);  
printf("x = %d\n\n",testPrint);
```

```
x = 10  
  
x = 10  
  
x = 10
```

Printf Personnalisé

Rendu personnalisé dans la console

```
printf("*****\n x = %d\n*****\n",testPrint);
```

```
*****  
x = 10  
*****
```

Print Raylib :

La fonction **DrawText()** de raylib peut prendre en paramètre une fonction de formatage qui est l'équivalent du printf et qui est la suivante : **TextFormat()**. Ci-dessous l'image du code complet qui sera dans l'exercice.

```
DrawText(TextFormat("x = %d\ny = %f\n",testPrint, testFloat), 190, 200, 20, LIGHTGRAY);
```

Vous noterez que les deux variables et la syntaxe sont présentes dans un unique appel. Il faut toutefois faire plusieurs retours à la ligne selon la taille de la police ce n'est pas géré automatiquement. Sinon vous pouvez faire deux appels **DrawText()**. Mais vous devrez alors changer les coordonnées Y et/ou X pour chaque appel supplémentaire.

Et oui le **TextFormat()** est le **printf** de [Raylib](#) et partage le même formatage.

✓ Résumé

- Point clé 1 printf permet d'afficher des variables formatées dans la console.
- Point clé 2 Utiliser des specifiers comme %d, %f pour matcher les types.
- Point clé 3 TextFormat est l'équivalent dans raylib pour DrawText.

🔧 Partie pratique :

Il est relativement simple : il s'agira d'afficher les valeurs des différentes variables suivantes avec au moins une ligne pour chaque variable. L'output (ou le print dans ce cas) peut soit être fait dans la console ou avec le print de raylib, étant donné que le principe de formatage avec l'option **TextFormat()** est le même. Le challenge sera de **printf** les deux variables avec un seul appel de la fonction.

Voici le screen de la correction

```
main.cpp x
rc > main.cpp > main(void)
1  #include "raylib.h"
2  #include <stdio.h>
3
4  int main(void)
5  {
6      // create a windows for the app with size and title as param
7      int testPrint = 10;
8      float testFloat = 5.5f;
9      InitWindow(800, 450, "raylib [core] example - basic window");
10     printf("x = %d\n\n",testPrint);
11     printf("y = %f\n\n",testFloat);
12     while (!WindowShouldClose())
13     {
14         BeginDrawing();
15         // need to clear the background every frame see the tutorial on the hello world section
16         ClearBackground(RAYWHITE);
17         DrawText(TextFormat("x = %d\ny = %f\n",testPrint, testFloat), 190, 200, 20, LIGHTGRAY);
18         EndDrawing();
19     }
20     CloseWindow();
21
22     return 0;
23 }
```

💡 À retenir :

- Attention : Le type du specifier doit correspondre au type de la variable, sinon vous aurez un comportement imprévisible
- Pensez toujours au \n pour une meilleure lisibilité



Validation des compétences

- L'élève sait inclure et utiliser printf correctement.
- L'élève sait formater des prints avec specifiers.
- L'élève comprend la différence entre console et raylib prints.



Définitions :

- [Raylib](#) : framework léger de dev de jeu. Open-source, multiplateforme et multilangages (c++,c, c#,lua, etc....)
- [Cstdio](#): Lecture et écriture sur la console
- Bibliothèque : Une bibliothèque de programmation est un ensemble de code pré-écrit, tel que des fonctions et des classes, que les développeurs peuvent utiliser pour accomplir des tâches spécifiques dans un programme sans avoir à réinventer la roue



Méthode/fonction :

- **Printf()** : pour afficher des textes formaté dans la console voir source [ici](#)
- **DrawText()** : permet d'afficher du texte dans la fenêtre
- **TextFormat()** : pour mettre en forme le **DrawText()** et lire une variable avec ce dernier